

I: Fair Workload Distribution

Source file name: fairworkload.c, fairworkload.cpp, fairworkload.java, or fairworkload.py

Author: Edwin Niño

The logistics department of **ByteCorp** employs N workers, numbered consecutively from 1 to N . To handle incoming delivery requests across D consecutive days, management implements a daily **round-robin** assignment scheme designed to distribute the workload as fairly as possible.

On day i ($1 \leq i \leq D$), a total of M_i tasks need to be completed. To prevent favoritism, the system randomly selects a starting worker k_i ($1 \leq k_i \leq N$). The M_i tasks are then allocated one by one as follows:

- The first task is assigned to worker k_i .
- In general, after worker x receives a task, the next task is assigned to worker $(x \bmod N) + 1$ (that is, worker $x + 1$ if $x < N$, or worker 1 if $x = N$).

This process continues sequentially until all M_i tasks of day i have been assigned. Note that this scheme guarantees that within any single day, the difference in the number of tasks assigned to any two workers is at most 1.

At the end of the season, management wants to evaluate the accumulated workload. Given the log of the D days, you are asked to answer Q performance queries of the form:

“How many total tasks did worker w complete across days 1 through d (inclusive)?”

For example, consider $N = 5$ workers across $D = 3$ days:

- On day 1, $M_1 = 7$ tasks are assigned starting with worker $k_1 = 2$. The tasks are allocated in sequence to workers 2, 3, 4, 5, 1, 2, and 3. In total, workers 2 and 3 complete 2 tasks each, while workers 1, 4, and 5 complete 1 task each.
- On day 2, $M_2 = 3$ tasks are assigned starting with worker $k_2 = 4$, which are allocated to workers 4, 5, and 1. Workers 2 and 3 receive 0 tasks on this day.
- On day 3, $M_3 = 10$ tasks are assigned starting with worker $k_3 = 1$. Because $10 = 2 \times 5$, two full rounds are completed and every worker receives exactly 2 tasks.

If a query asks for the total tasks completed by worker 2 up to day 1 ($d = 1, w = 2$), the answer is 2. For worker 2 across days 1 through 2 ($d = 2, w = 2$), the answer is $2 + 0 = 2$. For worker 4 across days 1 through 2 ($d = 2, w = 4$), the answer is $1 + 1 = 2$. Finally, for worker 5 across days 1 through 3 ($d = 3, w = 5$), the total accumulated tasks is $1 + 1 + 2 = 4$.

Input

The input consists of several test cases.

The first line of each test case contains three integers N , D , and Q ($1 \leq N \leq 10^9$, $1 \leq D \leq 100\,000$, $1 \leq Q \leq 100\,000$) --- the number of workers, the number of days, and the number of queries, respectively.

The next D lines describe the daily assignments. The i -th of these lines ($1 \leq i \leq D$) contains two integers k_i and M_i ($1 \leq k_i \leq N$, $0 \leq M_i \leq 10^9$) --- the identifier of the starting worker and the number of tasks to be distributed on day i .

The next Q lines describe the queries. Each line contains two integers d and w ($1 \leq d \leq D$, $1 \leq w \leq N$) --- representing the day threshold d and the worker ID w .

The input is terminated by a line containing three zeros (0 0 0). This line should not be processed.

It is guaranteed that the sum of D and the sum of Q across all test cases in the input will not exceed 500 000.

The input must be read from standard input.

Output

For each query, print a single line with an integer: the total number of tasks assigned to worker w from day 1 up to day d , inclusive.

The output must be written to standard output.

Sample Input	Sample Output
5 3 4	2
2 7	2
4 3	2
1 10	4
1 2	
2 2	
2 4	
3 5	
0 0 0	